

An Efficient UAV Localization Technique Based on Particle Swarm Optimization

Weizheng Zhang^{ID}, *Member, IEEE*, and Wei Zhang^{ID}, *Fellow, IEEE*

Abstract—Unmanned aerial vehicles (UAVs) have recently attracted tremendous attentions in both industries and academic communities. Thanks to the high mobility and flexibility, UAVs can be deployed in many scenarios to provide various types of services. In these scenarios, the position of the UAVs must be timely and accurately acquired to avoid UAV collisions and realize millimeter-wave beamforming. Particle swarm optimization (PSO) is a potential approach to fulfill localization under GPS-denied environment. However, it has the drawbacks of high complexity and relative large localization error. In this article, we consider the UAV localization problem based on improved PSO, which aims at reducing complexity and localization error. We firstly analyze the performance metrics and performance bounds of conventional PSO in the considered UAV localization scenario. Then, the particle initialization process is reconsidered, where a particle and search space reduction method is introduced as the hierarchical PSO (HPSO). Next, the particle updating schemes are redesigned based on the particle number, where the reference best particle is introduced to deal with the limitations in conventional PSO, this is called reference PSO (RPSO). Lastly, the proposed HPSO and RPSO are validated in simulation results. It is shown that the proposed PSO method has both reduced complexity and localization error compared with conventional PSO and other reference methods.

Index Terms—Localization, particle swarm optimization, UAV network.

I. INTRODUCTION

IN THE past decade, unmanned aerial vehicles (UAVs) have been commercialized as drones for aerial photography, flying base stations (BSs) for emergency communication coverage, surveillance aircrafts for battlefield deployment, and specialized drones for exclusive purposes [1], [2]. In academia, UAVs have been extensively studied in fields like computer vision, dynamic modelling, control, navigation, etc, where UAVs are deployed as signal sensing and processing units [3]. Recently, due to the

mobility, scalability and flexibility of varied types of UAVs [4], the utilization of UAVs in Internet of Things (IoT) [5] and wireless communication [6] have attracted tremendous attentions and interests. UAVs are regarded as a potential type of devices deployed in future communication system as flying wireless relays, BSs and users [7], [8]. Moreover, the development of millimeter-wave (mmWave) communication makes it favorable to combine mmWave antenna arrays in small UAVs [9], so that the data rate of UAV communication can be further improved by designing proper mmWave beamforming [10], [11].

In most of the UAV researches and applications, the geometrical location of UAVs are often needed in order to acquire and process the dedicated information [12]. Specifically, in UAV mmWave communication where UAVs act as BSs or user equipments (UEs), the spatial location of UAV is needed in designing the directional analog beamformer so that the mmWave signal from the transmitter is maintained to the physical line of sight (LoS) direction toward the receiver [13]. When UAVs are deployed as drone swarm members or flying cameras for battlefield formation, surveillance, or simultaneous localization and mapping (SLAM), the UAV positions are required in formation keeping, collision avoidance and data collection [14], [15]. When UAVs are deployed in flying ad hoc networks (FANETs) in multi-hop emergency communication, the localization of UAV nodes in the network are required to maintain reliable links, especially when the network topology frequently changes due to UAV mobility [16].

Therefore, to obtain the positions of UAVs, localization methods and algorithms are needed. The global positioning system (GPS) is mostly utilized. However, in some scenarios such as disasters or ultra dense urban environment, GPS may become unavailable to provide positioning service. Moreover, it is evaluated to have an average localization error of several meters due to multiple sources of error [17]. Additional mobility of UAV will deteriorate the localization error to 10-40 m, which is obviously not suitable for massive UAVs or drone swarm network [18]. To alleviate the error of GPS, other methods and techniques have been proposed. The differential GPS (DGPS) can eliminate some sources of error in GPS by the differential calculation of reference station and mobile station signals, but the error in mobile case is larger than that in static one [19]. Computer vision (CV) based methods like visual odometry and SLAM can be utilized in UAV cameras, but the hardware cost and computation load are much larger to obtain higher accuracy, which is not suitable for power limited UAVs [20]. Other methods like ultra wideband (UWB) make use of relative

Manuscript received 15 February 2022; revised 25 April 2022; accepted 16 May 2022. Date of publication 27 May 2022; date of current version 19 September 2022. This work was supported in part by the Key Area Research and Development Program of Guangdong Province under Grant 2020B0101110003, in part by the Shenzhen Science and Innovation Fund under Grant JCYJ20180507182451820, and in part by the Australian Research Council's Project funding scheme under Grant LP160101244. The review of this article was coordinated by Dr. Boya Di. (*Corresponding author: Wei Zhang.*)

Weizheng Zhang is with the School of Electrical Engineering and Intelligentization, Dongguan University of Technology, Dongguan 523830, China, and also with the School of Electrical Engineering and Telecommunications, University of New South Wales, Sydney, NSW 2052, Australia (e-mail: zzw@ieee.org).

Wei Zhang is with the School of Electrical Engineering and Telecommunications, University of New South Wales, Sydney, NSW 2052, Australia (e-mail: w.zhang@unsw.edu.au).

Digital Object Identifier 10.1109/TVT.2022.3178228

distance measurement and signal processing methods to localize the UAVs, but the flip ambiguity (FA) cannot be ignored due to the unavoidable measurement and environment noise between the measured and real position [21]. These aforementioned approaches and techniques are not suitable when UAVs in the network require high localization accuracy and low computation complexity.

In recent years, swarm intelligence (SI) algorithms have been widely studied in analyzing the collective behaviors of decentralized, self-organized systems [22]. They have been applied in unmanned vehicle and nano-robot control, data mining and cluster analysis [23]. The concept of SI is a population set of agents interacting with others inside a dedicated region. The agents follow simple rules and update their positions according to this rule [24]. Among many types of SI algorithm, bio-inspired computation (BIC) and evolutionary algorithm (EA) are mostly used. Ant colony optimization (ACO) is a typical BIC technique to simulate ant colonies finding food throughout a graph or map [25]. The robustness and scalability of ACO is favourable, but the computation load of ACO is higher than others. Particle swarm optimization (PSO) is another typical method, where an objective function is optimized by particles interacting from both social and cognitive behaviours [26]. The robustness and flexibility of PSO outperforms ACO in some scenarios with reduced complexity. The fast development of BIC methods has witnessed an explosive growth of new algorithms like artificial fish swarm algorithm (AFSA) [27], artificial bee colony (ABC) [28], cuckoo search (CS) [29], beetle antennae search (BAS) [30], etc. These algorithms are efficient in optimizing complicated functions, but the complexity is often increased by introducing more functionalities in the algorithm. Besides, Machine learning (ML), as a part of artificial intelligence (AI), can be utilized to solve the UAV sensing and resource management problems [31]. For example, in UAV sensing problem, the trajectory of UAV must be carefully designed to make sure it moves toward the expected location [32], where the positioning and localization between UAV and the target are important. Reinforcement learning (RL) based UAV positioning and resource management has been discussed in [33], which is a promising method in solving UAV target sensing and trajectory control. RL is expected to enhance the system performance especially when the UAV is operating in a highly dynamic and uncertain environment, which has great opportunities to be further investigated.

The high mobility makes UAV localization a challenging problem where the trade-off between localization error and computation complexity must be carefully considered and balanced. PSO, however, can be a suitable choice due to its efficiency and flexibility. There have been some researches on utilizing PSO to localize or estimate UAV positions in recent years. In [16] the authors considered bounded PSO in UAV network node localization and clustering, where the localization error was lower than other methods such as ACO and genetic algorithm (GA), but the complexity of bounded PSO has not been studied. The performance of conventional PSO has been analyzed in [26], where the convergence behaviour of particles and the overall convergence are analyzed. PSO has been improved or redesigned in many works, but few of them considered the utilization of PSO in UAV localization. Therefore, in this article, we consider the

UAV localization based on improved PSO. The main contributions of this work are listed as follows:

- 1) We firstly analyze the performance of conventional PSO in UAV localization system, where we define the particle efficiency (PE) to model the complexity of PSO performance in UAV localization. Then, we find that the initialization of particles in considered search region is contradictory to the favoured target of reducing PSO complexity, so the particle update criteria needs to be redesigned accordingly. Lastly, we briefly analyze the performance of PSO by following the result in [16], where we conclude several findings and objectives to design the particle update rule.
- 2) We propose a hierarchical PSO (HPSO) method to reduce the total computation complexity in conventional PSO, where several iterations in the initialization process are deployed to reduce the particle number deployed in the overall iterations. The HPSO is designed together with search space reduction, where the search space of particles is reduced at the end of each iteration. The proposed HPSO is validated and shown to have higher PE and narrower search space than conventional PSO, which is in line with the findings in the performance analysis.
- 3) Furthermore, we modify the particle update criteria based on the particle number deployed. When the particle number is insufficient, we design the update rule by finding a reference best position. This RPSO-I method aims at enhancing the global search ability by comparing the sub-fitness functions of particles. The reference best position can substitute the combination of local best and global best in conventional PSO. The simulation results show that the proposed RPSO-I outperforms standard PSO and other reference methods in both the average localization error and the statistical result.
- 4) When the number of particles is sufficient or relatively large, we modify the PSO update rule by RPSO-II, where we combine the particle fitness value with a predefined threshold. When the target number is below half of used particles, we focus on the global search, and when the number is larger, local search ability is maintained. By tuning the coefficient of social and cognitive coefficients, we update the reference best position by the renewed combination of local and global best. Finally, a weighted average is used for the particle update. Simulation results also validate the effectiveness of proposed method.

The rest of this article is organized as followed. Section II introduces our system model. In Section III we present the UAV swarm localization methods based on proposed particle swarm optimization. Simulation results are shown in Section IV followed by the conclusion in Section V.

Notations: Bold uppercase letter $\mathbf{X}$ represents matrices, bold lowercase letter $\mathbf{x}$ represents vectors. Mathcal uppercase letters $\mathcal{X}$ represents sets of numbers, and uppercase greek letters represent sets of vectors or matrices. $\mathcal{X} \setminus x$ means removing one element x from set $\mathcal{X}$. $[\cdot]^T$ represents transpose of vectors or matrices. $[\cdot]^{-1}$ represents inverse of vectors or matrices. Special operator defined in this article is denoted as $\oplus$. Expectation operation of random variables is denoted as $\mathbb{E}[\cdot]$, and the union set operation is denoted by $\cup$.

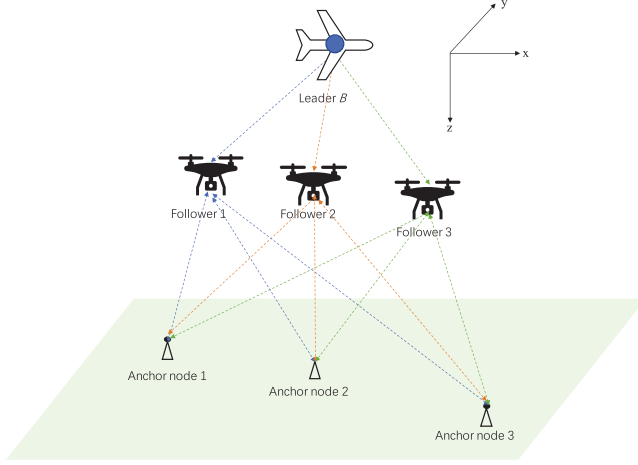


Fig. 1. System Model.

II. SYSTEM MODEL

We consider a UAV air-to-air (A2A) model, where one fixed-wing UAV B is linked with a single or multiple aerial users, as shown in Fig. 1. The UAV B is also a leader UAV sending formation orders to the UAV followers. The UAV leader B has capability to process the localization information. Besides, the leader UAV is assumed to have relatively fixed position in the 3-dimensional space. The followers have limited computation capability so that they have no knowledge of the position. Besides, we assume that the GPS system is unavailable. Therefore, the UAV leader is conducting localization, formation control, and communication through the A2A link. On the other hand, UAV followers only feed back assisting information to the leader, such as measured relative distance. To assist the leader localizing the follower, some anchor nodes are also deployed. The relative distance to UAV follower is measured by receive signal strength indicator (RSSI).

A. Geometrical Model

We take the barycenter of B as the origin $\mathbf{p}_B = [0, 0, 0]^T$ of geometrical coordinate system $O_G \in \mathcal{G}^{3 \times 1}$, where

$$\mathcal{G} = \{[x, y, z]^T\} = \left\{ \begin{array}{l} |x| \leq X_B \\ |y| \leq Y_B \\ |z| \leq Z_B \end{array} \right\} \quad (1)$$

is the considered space covered by UAV B , and X_B, Y_B, Z_B are the boundary values of x-axis, y-axis, z-axis, respectively. The speed of UAV follower k is then denoted as a vector $\mathbf{v}_{k,t} = [v_{x,k,t}, v_{y,k,t}, v_{z,k,t}]^T$ at time slot $t, t = 1, 2, \dots, T$. The general mobility model of UAV follower k can be represented as:

$$\mathbf{v}_{k,t} = \omega_v \mathbf{v}_{k,t-1} + \varepsilon_{k,t} \quad (2)$$

where ω_v is the inertia coefficient, $\varepsilon_{k,t}$ is the variation of speed, which is zero for uniform motion, fixed for uniformly acceleration motion, and random for unknown motion, respectively. Furthermore, the displacement model can be represented as:

$$\mathbf{p}_{k,t} = \mathbf{p}_{k,t-1} + \mathbf{v}_{k,t} \Delta_t + \chi_{k,t} \quad (3)$$

where $\mathbf{p}_{k,t}$ is the position of UAV follower k in time slot t , Δ_t is the time stamp of one slot, $\chi_{k,t}$ represents the measured noise, which can be regarded as the passive mobility caused by wind, controller error or shaking, vibrating, etc.

B. UAV Localization via Trilateration

We consider the UAV leader B localize the followers sequentially at one time slot. Therefore, the user index k and time slot t are omitted for simplicity. To obtain the coordinates of UAV followers in the covered space (1), we assume that the relative distance from followers to the leader and the distance from followers to anchor nodes are stored in UAV leader already. In detail, we consider L anchor nodes utilized in the $z = 0$ plane covered by UAV leader. Hence, each anchor node has fixed and known position, shown in Fig. 1. The coordinates of UAV follower $\mathbf{p}_f = [x_f, y_f, z_f]^T$ will be estimated. The position of the l -th anchor node is denoted as $\mathbf{p}_l = [x_l, y_l, z_l]^T$, $l = 1, 2, \dots, L$. According to the geometrical expression, the relative distance between UAV follower and anchor nodes are denoted as:

$$\begin{cases} d_1^2 = (x_1 - x_f)^2 + (y_1 - y_f)^2 + (z_1 - z_f)^2 \\ d_2^2 = (x_2 - x_f)^2 + (y_2 - y_f)^2 + (z_2 - z_f)^2 \\ \vdots \\ d_L^2 = (x_L - x_f)^2 + (y_L - y_f)^2 + (z_L - z_f)^2 \end{cases} \quad (4)$$

Use the last row of above equation and subtract every other $L - 1$ rows, we can get the exact coordinates of UAV follower using least-square (LS) method:

$$\mathbf{p}_f = [x_f, y_f, z_f]^T = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{B} \quad (5)$$

where

$$\mathbf{A} = \begin{bmatrix} 2(x_L - x_1) & 2(y_L - y_1) & 2(z_L - z_1) \\ 2(x_L - x_2) & 2(y_L - y_2) & 2(z_L - z_2) \\ \vdots & \vdots & \vdots \\ 2(x_L - x_{L-1}) & 2(y_L - y_{L-1}) & 2(z_L - z_{L-1}) \end{bmatrix} \quad (6)$$

and

$$\mathbf{B} = \begin{bmatrix} x_L^2 - x_1^2 + y_L^2 - y_1^2 + z_L^2 - z_1^2 + d_1^2 - d_L^2 \\ x_L^2 - x_2^2 + y_L^2 - y_2^2 + z_L^2 - z_2^2 + d_2^2 - d_L^2 \\ \vdots \\ x_L^2 - x_{L-1}^2 + y_L^2 - y_{L-1}^2 + z_L^2 - z_{L-1}^2 + d_{L-1}^2 - d_L^2 \end{bmatrix} \quad (7)$$

Therefore, the coordinates of follower can be obtained if the relative distance $d_l, l = 1, 2, \dots, L$ are obtained. This method is called Trilateration.

III. UAV LOCALIZATION BASED ON PARTICLE SWARM OPTIMIZATION

With the Trilateration method, the UAV follower position could be calculated if the relative distance d_l is known. However, the distance is usually measured by methods such as received signal strength indicator (RSSI), which will bring measurement

error, leading to position estimation deviation. This is unacceptable if UAVs have strict collision avoidance requirements in formation keeping. In this section, we solve the localization problem via particle swarm optimization (PSO). Firstly, a brief summary of conventional PSO will be presented. Then, enhanced PSO methods will be proposed for UAV localization.

A. PSO Based UAV Localization

PSO is a heuristic searching method to find global optima by moving the virtually deployed particles. The speed and position of particles are updated accordingly. A standard PSO has several procedures. To deploy it in a UAV localization scenario, we specify it as the following steps.

1) *Initialization*: First of all, the leader will initialize all particles in the search space, i.e. R particles are randomly deployed in the D -dimensional space, take $D = 3$ here as an example. Each particle will have a position $\mathbf{p}_n = [x_n, y_n, z_n]^T$, $n = 1, \dots, R$. Then, each particle generates a random speed vector $\mathbf{v}_n = [v_{x,n}, v_{y,n}, v_{z,n}]^T$, where $v_{x,n}, v_{y,n}, v_{z,n}$ are i.i.d. uniformly distributed random values with $\mathcal{U}(-1, 1)$. The iteration time N is predetermined. The particles will “fly” to an updated direction at each iteration. Hence, the direction updates of each particle is the key process of PSO algorithm.

2) *Calculate Fitness Function Value*: The fitness function is used to quantify the “closeness” between a particle’s current position and the actual position of target UAV follower. The target has its real coordinates as $\mathbf{p}^{UE} = (x, y, z)$, which is unknown. Hence, the fitness function is actually a function of $f(x, y, z) = f(\mathbf{p}^{UE})$. Here, we utilize the sum of squared distance error as the fitness function:

$$\begin{aligned} \text{fitness}(\mathbf{p}^{UE}) &= |d - d_{\text{mea},B}|^2 + \sum_{l=1}^L |d_l - d_{\text{mea},l}|^2 \\ &= \sum_{m=1}^{M=L+1} g(\Delta_m) \end{aligned} \quad (8)$$

where $d = \sqrt{x^2 + y^2 + z^2}$ is the relative distance from target UAV follower to UAV leader, $d_l = \sqrt{(x - x_l)^2 + (y - y_l)^2 + (z - z_l)^2}$ is the relative distance from target UAV follower to the l -th anchor node, $d_{\text{mea},B}$ is the measured relative distance between UAV follower and UAV leader, and $d_{\text{mea},l}$ is the measured relative distance between UAV follower and the l -th anchor node. $g(\Delta_m)$ represents the m -th of $M = L + 1$ sub-fitness functions of this distance error. We assume that $d_{\text{mea},B}$ and $d_{\text{mea},l}$ are already known by RSSI measurement. However, this relative distance is usually not error-free, making the estimation of UAV follower’s location inaccurate. Hence, to model this error in measurement, we use the zero mean Gaussian noise represented as:

$$d_{\text{mea},B} = d + \epsilon_B \quad (9)$$

$$d_{\text{mea},l} = d_l + \epsilon_l \quad (10)$$

where $\epsilon_B \sim \mathcal{N}(0, \sigma_B^2)$, $\epsilon_l \sim \mathcal{N}(0, \sigma_l^2)$ represent the noise variance, respectively.

3) *Search for Personal Best Value*: After determining the fitness function, the particles will start iteration. At each iteration time l , $l = 1, \dots, N$, each particle will move toward an updated direction which is according to the particle’s personal best value and all particles’ global best value. Start with the personal best value denoted as $pbest_{n,l}$ of particle n in iteration time l , it means the minimum fitness value of this particle among all l finished iterations:

$$pbest_{n,l} \triangleq \mathbf{p}_{n,l} = \min_{\mathbf{p}_{n,j^*}} \{j | f(\mathbf{p}_{n,j}), j = 1, 2, \dots, l\} \quad (11)$$

where $\mathbf{p}_{n,j} = [x_{n,j}, y_{n,j}, z_{n,j}]^T$ is the 3×1 coordinate vector of particle n at j -th iteration time. For the initial iteration $j = 1$, the personal best of each particle is set equal to itself.

4) *Search for Global Best Value*: The position of target UAV follower is to be estimated. We can use the particles at iteration time l to obtain a set of fitness values by taking the particle’s coordinates into (8). Then, a set of R fitness values for at iteration l is obtained, written as:

$$\mathfrak{F}_l = \{f(\mathbf{p}_{1,l}), f(\mathbf{p}_{2,l}), \dots, f(\mathbf{p}_{R,l})\} \quad (12)$$

which is a set of distance errors from particle to UAV leader B and anchor nodes. The global best value is defined as the particle in current iteration l who has the minimum fitness value. Hence, it can be represented as:

$$gbest_l \triangleq \mathbf{g}_{\mathbf{b}_l} = \min_{\mathbf{p}_{n^*} \in \{1, 2, \dots, R\}, l} \mathfrak{F}_l \quad (13)$$

5) *Update and Output*: After finding $pbest_{n,l}$ and $gbest_l$, the new velocity of particle n can be updated in iteration time l by the following criterion:

$$\begin{aligned} \mathbf{v}_{n,l+1} &= \mathbf{v}_{n,l} + c_1 \mathbf{r}_{1,l} (\mathbf{p}_{n,l} - \mathbf{p}_{n,l}) \\ &\quad + c_2 \mathbf{r}_{2,l} (\mathbf{g}_{\mathbf{b}_l} - \mathbf{p}_{n,l}) \end{aligned} \quad (14)$$

where c_1 and c_2 are constants called cognitive and social coefficient, respectively. $\mathbf{r}_{1,l}$ and $\mathbf{r}_{2,l}$ are 3×1 random vectors where each element is a standard uniformly distributed random variable in $U(0, 1)$. In order to control and adjust the flying velocity of particles, an inertia weight ω decreasing linearly from 0.9 to 0.4 is considered by [26], the velocity can be then updated by:

$$\begin{aligned} \mathbf{v}_{n,l+1} &= \omega \mathbf{v}_{n,l} + c_1 \mathbf{r}_{1,l} (\mathbf{p}_{n,l} - \mathbf{p}_{n,l}) \\ &\quad + c_2 \mathbf{r}_{2,l} (\mathbf{g}_{\mathbf{b}_l} - \mathbf{p}_{n,l}) \end{aligned} \quad (15)$$

The velocity can be also updated by the assistance of introducing a restriction factor χ :

$$\begin{aligned} \mathbf{v}_{n,l+1} &= \chi [\mathbf{v}_{n,l} + c_1 \mathbf{r}_{1,l} (\mathbf{p}_{n,l} - \mathbf{p}_{n,l}) \\ &\quad + c_2 \mathbf{r}_{2,l} (\mathbf{g}_{\mathbf{b}_l} - \mathbf{p}_{n,l})] \end{aligned} \quad (16)$$

where χ is usually set as constant.

Then, the updated position of particle n in iteration time l is:

$$\mathbf{p}_{n,l+1} = \mathbf{p}_{n,l} + \mathbf{v}_{n,l+1} \quad (17)$$

Now, particles’ position changes to $\mathbf{p}_{n,l+1}$. The fitness values, personal best, global best will be updated. Then, the new velocity and position will be updated for iteration $l + 1$ repeatedly. After N iteration times, in the last update, the processing unit will

Algorithm 1: Standard Particle Swarm Optimization for UAV Localization.

- 1: **Input:** Swarm size R , velocity bound v_{max}, v_{min} , cognitive coefficient c_1 , social coefficient c_2 , iteration time N , dimension D , search region bound X_B, Y_B, Z_B , measured relative distance $d_{mea,BS}, d_{mea,l}$, $l = 1, 2, \dots, L$
 - 2: **Output:** Estimated location coordinates of UAV follower
 - 1) Initialization R particle positions $\mathbf{p}_{n,0}, n = 1, \dots, R$ within region bound, and velocity vectors $\mathbf{v}_{r,0}$ within velocity bound.
 - 2) For $n = 1 : R$, calculate the fitness function values of each particle $f(\mathbf{p}_{n,0})$ and set $\mathfrak{F}_0$.
 - 3) Initialize the global best $\mathbf{gb}_0$ according to (13).
 - 4) For $n = 1 : R$, initialize the personal best $\mathbf{pb}_{n,0}$ same as initial positions, i.e. $\mathbf{pb}_{n,0} = \mathbf{p}_{r,0}$
 - 5) For iteration $l = 1 : N$, do
 - a) Update the velocity vector of each particle according to (14)
 - b) Each particle moves to the new position according to (17)
 - c) For $n = 1 : R$, calculate the fitness function values of each particle's new position
 - d) Update global best $\mathbf{gb}_l$ if $f(\mathbf{gb}_l) < f(\mathbf{gb}_{l-1})$
 - e) For $n = 1 : R$, update personal best $\mathbf{pb}_{n,l}$ if $f(\mathbf{pb}_{n,l}) < f(\mathbf{pb}_{n,l-1})$
 - f) iteration +1.
 - 6) Output the global best value $\mathbf{gb}_N$ as the estimated location of UAV follower
-

find the global best particle and this position will be the output of estimated position of target UAV follower.

$$\hat{\mathbf{p}} = \min_{\mathbf{p}_{n^*} \in \{1, 2, \dots, R\}, N} \mathfrak{F}_N \quad (18)$$

The general standard PSO algorithm is concluded by Algorithm 1.

B. Performance Analysis of PSO Based Localization

In order to improve the PSO performance, there are two main aspects. One is to combine with other swarm intelligence algorithms, such as genetic algorithm (GA), ant colony optimization (ACO), etc. The other is to improve the particle update by designing of ω or χ in (15) (16), and use modified particle update, such as hierarchy order by finding neighborhood structure of particles. However, they lack consideration on three factors: UAVs have limited computation capability, the PSO performance is greatly affected by the initialization process, and the converging PSO global best in UAV localization system does not guarantee satisfactory result. Therefore, we will firstly analyze the PSO performance from the perspective of computation complexity, particle initialization and convergence performance, respectively.

1) *PSO Complexity Analysis:* For the conventional PSO described above, a major problem is the complexity. In order to

achieve K UAV followers' localization, KR particles will be deployed and KN iterations will be needed. The computation complexity is too high when K or R goes large. On the other hand, the computation load for one UAV follower is also high. For example, if $R = 200$ particles are initialized to localize a UAV follower in a $100m \times 100m$ 2-dimensional space, after $N = 50$ iterations, the total computations of fitness functions will be $C_{total} = R(N + 1) \approx 10^4$. We define the particle efficiency (PE) in iteration time l can be denoted as the best fitness over total particles consumed:

$$PE_l = \frac{\min_n \{f(\mathbf{p}_{n,l}), n = 1, \dots, R_l\}}{\sum_{j=1}^l R_j} \quad (19)$$

where the numerator is the best fitness in current iteration l , R_l is the particle number used in iteration l . The denominator is the summation of consumed particles, where R_j is the particles used in iteration j . For a standard PSO, $R_j = R_l = R$.

Therefore, low-complexity PSO design is required to reduce the total number of particles, which will decrease the computation load to find $pbest$ and $gbest$ instead of computing all particles' fitness values in each iteration. To achieve this, we will introduce HPSO in the next subsection, which is based on reducing the total number of particles hierarchically as well as eliminating the search space by bounding particle moving regions.

2) *Particle Initialization Performance Analysis:* The initialization of particles includes the step that randomly deploys R particles in search space. The particles at initialization step are independent, hence, the performance at initialization stage will be affected by the distribution of particles and the number of particles. For example, given an error bound U_X and U_Y in a 2 dimensional region, if R particles are initialized, there will be particles outside the bound region. Consider the worst case, i.e. R particles are i.i.d. distributed. Let $[x_o, y_o]$ represent the true position of UAV follower. Then, the probability that at least one particle is inside the bounding region including true position is:

$$P_{in} = 1 - P_{out}^R \quad (20)$$

where P_{out} is the probability that one particle is outside the bounding region. If uniformly distributed particles are considered, the probability is:

$$P_{out} = 1 - \frac{4U_X U_Y}{S_{xy}} \quad (21)$$

where $S_{xy} = X_B Y_B$ is the rectangular area covered by the UAV leader.

From (21) we can find that given the error bound U_x and U_y , the probability P_{in} is monotonically increasing with increased particle number R . For example, consider a case $X_B = Y_B = 100m$, $U_x = U_y = 5m$, then $P_{out} = 0.99$. To achieve a probability $P_{in} \geq 0.5$, at least 70 particles are needed. Moreover, we can find that when R is small, the probability P_{in} will be too small. In this case, the particles need more social evolutions, and global best value should be treated more. On the other

hand, when R is large, social evolutions should be paid less attention than cognitive evolutions, and personal best value should be treated more. Based on this conclusion, we focus on improving the PSO performance and proposing new particle update criteria for social and cognitive parameters. Note that the overall probability in iteration time $l = 1, 2, \dots, N$ is much more complicated due to the update rule of each particle, we will briefly analyze the convergence performance next.

3) *PSO Convergence Performance Analysis*: For the standard PSO update rule in (14), we can simplify the equation by considering a single dimensional case. This is because the dimensional positions do not interact with others, and each particle could be treated as independent until the global best gb and personal best pb change. In this way, we can rewrite (14) as:

$$v_{l+1} = v_l + \phi_1(pb - p_l) + \phi_2(gb - p_l) \quad (22)$$

where $\phi_1 = c_1 r_1$, $\phi_2 = c_2 r_2$ and the update rule is rewritten as:

$$p_{l+1} = p_l + v_{l+1} \quad (23)$$

Substituting (22) into (23), we can get:

$$\begin{aligned} p_{l+1} &= p_l + v_l + \phi_1(pb_l - p_l) + \phi_2(gb_l - p_l) \\ &= (1 - \phi_1 - \phi_2)p_l + \phi_1 pb_l + \phi_2 gb_l + v_l \end{aligned} \quad (24)$$

Consider $p_l = p_{l-1} + v_l$, we can get the non-homogeneous relationship as:

$$\begin{aligned} p_{l+1} &= (1 - \phi_1 - \phi_2)p_l + \phi_1 pb_l + \phi_2 gb_l + p_l - p_{l-1} \\ &= (2 - \phi_1 - \phi_2)p_l + \phi_1 pb_l + \phi_2 gb_l - p_{l-1} \end{aligned} \quad (25)$$

By using simple matrix formation and characteristic polynomial, the explicit form of (25) can be obtained:

$$p_l = t_1 + t_2 \alpha^l + t_3 \beta^l \quad (26)$$

where t_1, t_2, t_3 are constants. Solving (26), we can get:

$$t_1 = \frac{\phi_1 pb + \phi_2 gb}{\phi_1 + \phi_2} \quad (27)$$

$$t_2 = \frac{\beta(p_0 - p_1) - p_1 + p_2}{\gamma(\alpha - 1)} \quad (28)$$

$$t_3 = \frac{\alpha(p_1 - p_0) + p_1 - p_2}{\gamma(\beta - 1)} \quad (29)$$

where

$$\alpha = \frac{2 - \phi_1 - \phi_2 + \gamma}{2} \quad (30)$$

$$\beta = \frac{2 - \phi_1 - \phi_2 - \gamma}{2} \quad (31)$$

$$\gamma = \sqrt{(2 - \phi_1 - \phi_2)^2 - 4} \quad (32)$$

and p_0, p_1, p_2 are obtained by one initialization and two iterations. Note that γ could be an either real or complex constant since the value under squared root could be greater or smaller than zero, which is determined by the value of c_1 and c_2 . From this result, we can also find that c_1 and c_2 are important in deciding the PSO convergence.

Now, the convergence behaviour is analyzed. The particle position p_l can be computed by (26), as long as the assumption holds that ϕ_1, ϕ_2, pb, gb are constant. In this way, the convergence of p_l is determined by α and β . It is easily found that when $\max\{\|\alpha\|, \|\beta\|\} < 1$, p_l will converge, which also gives the guidance bound of designing c_1 and c_2 .

Moreover, if we relax the assumption to that $\phi_1 = c_1 r_1$ and $\phi_2 = c_2 r_2$, where r_1 and r_2 are randomly chosen from uniform distribution $U(0, 1)$. If the convergence holds, we can get the average convergence value as:

$$\lim_{l \rightarrow \infty} \mathbb{E}_l[p_l] = \mathbb{E}_l t_1 \quad (33)$$

$$= \mathbb{E} \left[\frac{\phi_1 pb + \phi_2 gb}{\phi_1 + \phi_2} \right] \quad (34)$$

$$\approx \frac{c_1 pb + c_2 gb}{c_1 + c_2} \quad (35)$$

$$= \kappa pb + (1 - \kappa) gb \quad (36)$$

where $\kappa = \frac{c_1}{c_1 + c_2}$. This result shows that the particle converges to a value derived from the line connecting the personal best to the global best. In the iteration steps, particles will search for better pb and gb in the area between its past personal best position and the current global best position. However, the particles may oscillate between personal best and global best if they cannot be stably updated.

In summary, from the above analysis, we can get the following findings:

- 1) To increase particle efficiency or reduce the computation load at UAV leader, reduced particle number is necessary.
- 2) The initialization of particles can be further considered to reduce the complexity from the perspective of search space and particle number.
- 3) To avoid particle oscillation, the particle update rule should be redesigned when particle number is insufficient.
- 4) To improve the localization performance further when distance measurement is not error-free, the particle update rule should be redesigned when the particle number is large.
- 5) The improved PSO design should hold the constraint that $\max\{\|\alpha\|, \|\beta\|\} < 1$, which determines the lower bound of choosing c_1 and c_2 .

Next, we will propose the improved PSO design. Firstly, finding 1 and finding 2 will be considered to reduce complexity. Then, finding 3 and finding 4 will be targeted respectively in accordance with finding 5.

C. Proposed PSO Design Part 1: Reducing Complexity With HPSO

From the above analysis, the initialization of particles could be improved via reducing particle number and search space. Therefore, we consider from two perspectives: one is the hierarchical particle initialization, which is to reduce the particle number for the beginning iterations. The other is the search space reduction, which is to reduce the search space at each iteration. The general idea can be briefly illustrated in Fig. 2, where $Q = 4$ iterations

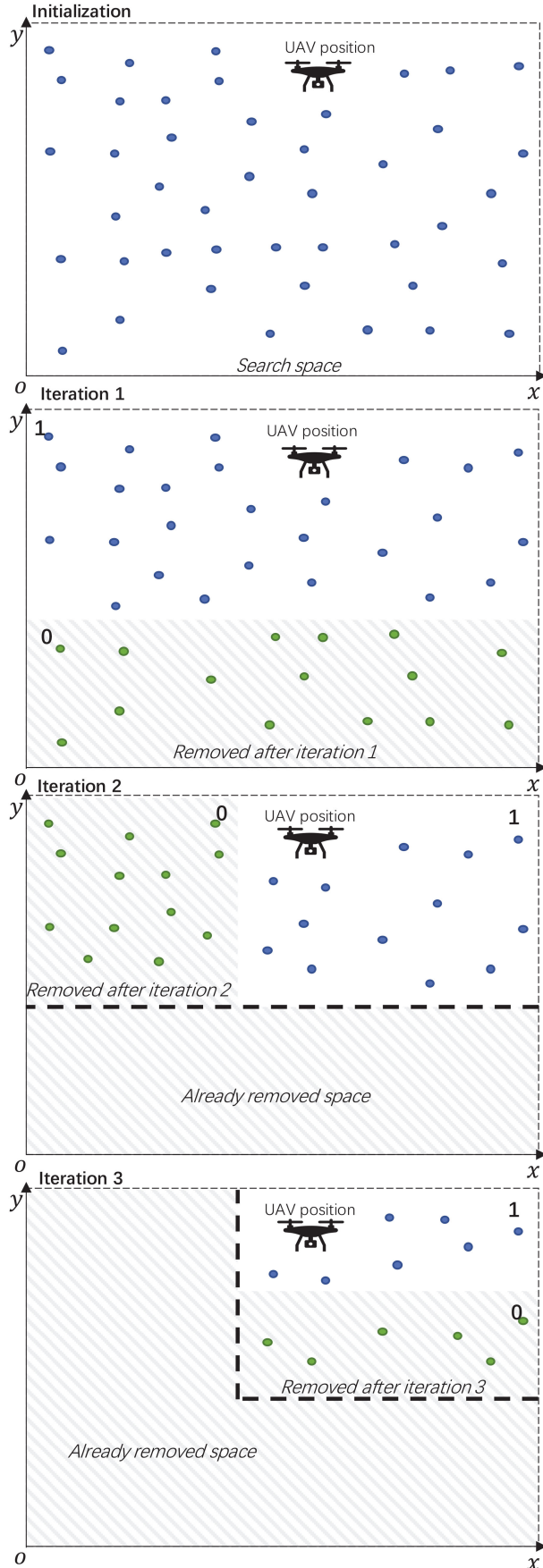


Fig. 2. Example of HPSO.

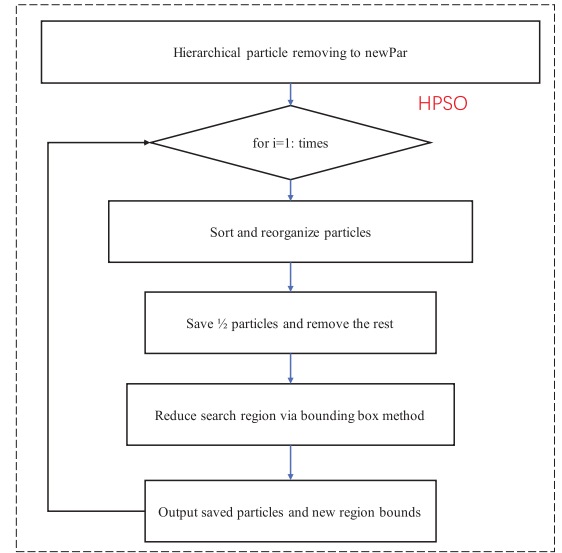


Fig. 3. Overall flow chart of HPSO.

are considered at the initialization stage. From iteration 1 to 4, the search region and particles are always divided into two parts. The particles inside the part will be reserved while the rest will be removed. Besides, the particles inside the other part but near the bound will also be reserved so that when UAV is close to the bound, the search region after each iteration will also be robust enough to include the possible area that UAV covers. In the rest $N - Q$ iterations, the particle number will not reduce, but the search space will still be updated according to particle bound and fitness value, which will be explained later. The combination of those two ideas is denoted as the hierarchical PSO (HPSO). The overall flowchart of HPSO is summarized as Fig. 3. Next we will describe each step in details.

1) *Step 1: Sort and Reorganize Particles*: The particle number R means there will be R fitness computation loads at each iteration. Reducing R directly reduces the computation complexity. Therefore, after the very first iteration, when R fitness values are found, the HPSO will firstly sort the fitness values in a descending order. For example, if we denote $f_n = \text{fitness}(\mathbf{p}_n)$ as the fitness function value of n -th particle in position $\mathbf{p}_n$, then, the set $F = \text{sort}(\{f_1, f_2, \dots, f_R\})$ is obtained. The first global best is found by $\mathbf{gb}_0 = \min_n f_n$. Each particle will be reorganized to an index 1 or 0, representing that it is with the first or second half of the search region, which is predefined before the initialization.

2) *Step 2: Save and Remove Particles*: Without loss of generality, assume that the particle corresponding to $\mathbf{gb}_0$ is with index 1, which means the minimum fitness appears in the first half search region. Then, all the particles with index 1 are saved. Now we compute the average fitness value $\bar{f} = \frac{\sum_{n=1}^R f_n}{R}$. If the number of saved particle $S < R/2$, then the smallest $R/2 - S$ particles marked as index 0 s will join in the group of saved particles. Here the $R/2 - S$ particles are chosen from the particles in the second half of search region, and they are with the smallest fitness values compared with others inside this region. On the other hand, if $S > R/2$, the $S - R/2$ particles

will also be removed, which means $S - R/2$ particles with index 1 corresponding with larger fitness function values will be removed from the group of saved particles. Then, the search region will be reduced and reallocated in next step according to the union bound of saved particles. After repeating Q iterations, the total number of particles needed in the following $L - Q$ iterations will be $\frac{R}{2^{Q-1}}$.

3) *Step 3: Reduce and Reallocate Search Space*: After removing and saving particles, the search region or space will be reconsidered. The saved particles in each of Q iterations will have an union bound together. This bound will be regarded as the search space in next iteration. And after Q iterations where particle number is removed to $\frac{R}{2^{Q-1}}$, the search space will still be updated by finding the union bound of $\frac{R}{2^{Q-1}}$ particles iteration l , where $l > Q$.

$$\begin{aligned} \Xi_l &= \bigcup_{n=1}^{\frac{R}{2^{Q-1}}} \mathbf{p}_{n,l} \\ &= \{(x, y, z) | x_{lo} \leq x \leq x_{hi}, y_{lo} \leq y \leq y_{hi}, z_{lo} \leq z \leq z_{hi}\} \end{aligned} \quad (37)$$

where the subscripts $\cdot_{lo}$ and $\cdot_{hi}$ is the lower and upper bound coordinate of each dimension, respectively.

D. Proposed PSO Design Part 2: Improving PSO Performance With Insufficient Particles

The HPSO is used to reduce the total number of particles used in each iteration. After decreasing the computation load for the first Q iterations, another perspective to enhance the performance is to improve the localization accuracy in the rest $N - Q$ iterations. Apart from the search space update in HPSO, we can also modify the particle update criteria to deal with the particle oscillation in finding 3. In detail, when the number of particles is not large, the particle will move between its personal historical best position and current global best position. The linear summation of personal best and global best in (14) leads to a result controlled by two random factors. Besides, from the above analysis, the cognition coefficient c_1 and social coefficient c_2 would affect or even destroy the convergence and accuracy of PSO localization. Earlier researches on comparing social-only and cognitive-only velocity update have found that social components are more possible to affect the results of PSO when the particle number is insufficient, because the global best is likely to change, which will deteriorate the particle oscillation. Therefore, we aim at the particle update in (14) in order to mitigate the particle oscillation. We can rewrite (14) as:

$$\begin{aligned} \mathbf{v}_{n,l} &= \mathbf{v}_{n,l-1} + (c_1 \mathbf{r}_1 \mathbf{p}_{n,l-1} + c_2 \mathbf{r}_2 \mathbf{g}_{n,l-1}) \\ &\quad - (c_1 \mathbf{r}_1 + c_2 \mathbf{r}_2) \mathbf{p}_{n,l-1} \\ &= \mathbf{v}_{n,l-1} + (c_1 \mathbf{r}_1 + c_2 \mathbf{r}_2) (\mathbf{r}_{b,n,l-1} - \mathbf{p}_{n,l-1}) \\ &\approx \mathbf{v}_{n,l-1} + c \mathbf{r} (\mathbf{r}_{b,n,l-1} - \mathbf{p}_{n,l-1}) \end{aligned} \quad (38)$$

where $\mathbf{r}_{b,n,l-1} = \frac{c_1 \mathbf{r}_1 \mathbf{p}_{n,l-1} + c_2 \mathbf{r}_2 \mathbf{g}_{n,l-1}}{c_1 \mathbf{r}_1 + c_2 \mathbf{r}_2}$ is defined as the reference best $\mathbf{r}_{best}$. The approximation is when $c = c_1 = c_2$. With this representation, the aim is to modify the conventional PSO

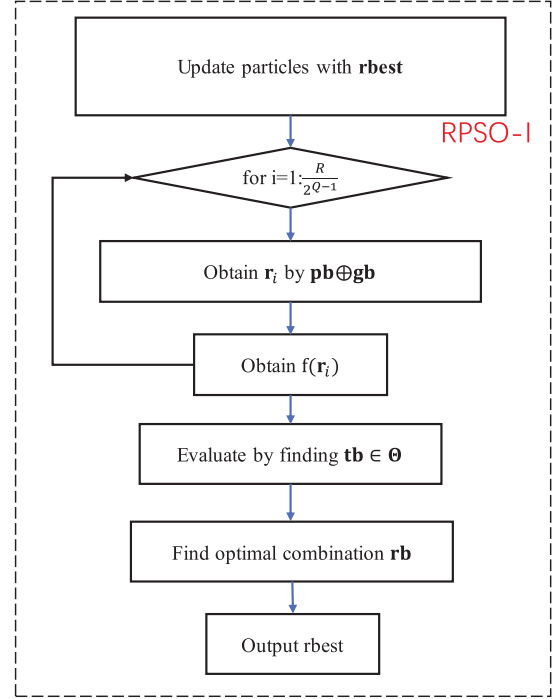


Fig. 4. Flowchart of RPSO-I.

and improve the particle update based on $\mathbf{r}_{best}$. We call this reference particle swarm optimization type one (RPSO-I). The flowchart of RPSO-I is summarized in Fig. 4. Next we will describe the design step-by-step.

Step 1. Obtain the Reference Indicator Set of 2^D Combinations: For the i -th particle, use the following operation:

$$\mathbf{t}_{b,n,l} = \mathbf{p}_{b,n,l} \oplus \mathbf{g}_{b,l} \quad (39)$$

to obtain a set of combinations from the personal best of particle i and the global best in current iteration. Here the operation $\oplus$ means the element-wise selection. For example, when $D = 2$, let $\mathbf{p}_{b,n,l} = [x_{p,n,l}, y_{p,n,l}]$ and $\mathbf{g}_{b,l} = [x_{g,l}, y_{g,l}]$, then, $\mathbf{t}_{b,n,l}$ is chosen from a set with five vectors corresponding with $2^D + 1$ possible combinations.

$$\begin{aligned} \Theta_{n,l} &= \left\{ [x_{p,n,l}, y_{p,n,l}], [x_{g,l}, y_{g,l}], [x_{p,n,l}, y_{g,l}], [x_{g,l}, y_{p,n,l}], \right. \\ &\quad \left. \left[\frac{1}{2}(x_{p,n,l} + x_{g,l}), \frac{1}{2}(y_{p,n,l} + y_{g,l}) \right] \right\} \end{aligned} \quad (40)$$

Step 2. Evaluation: After finding the set of reference indicators, an ergodic evaluation is required, where fitness values will be computed of each vector in the reference set. The vector corresponding to the minimum of the $2^D + 1$ fitness values is then marked as:

$$\mathbf{t}_{b,n,l} = \min_{i,j=1:2, i,j \in \{p,g\}} \{f([x_{i,n,l}, y_{i,n,l}]), [x_{i,n,l}, y_{i,n,l}] \in \Theta\} \quad (41)$$

This is when the global best is a satisfying localization result so that the $f(\mathbf{g}_{best})$ is small, which corresponds to a good estimation when the distance measurement is error-free.

However, when the measured distance is deteriorated by errors, the combination set is slightly changed. Denote $\theta = \frac{c_1 \mathbf{r}_1}{c_1 \mathbf{r}_1 + c_2 \mathbf{r}_2} \in [0, 1]$, $\mathbf{rb} = \theta \mathbf{pb} + (1 - \theta) \mathbf{gb}$. θ can be regarded as a random weighting factor. In the case when the distance measurement error is large, we use the $L + 1$ sub-functions defined in (8) to evaluate the performance of reference indicators. For the d -th vector in the set of 2^D vectors in Θ , we firstly let $g_{n,l,m} = g_m(\mathbf{p}_{n,l})$ represent the m -th sub-fitness value of particle n in iteration l , $n = 1, 2, \dots, R$, $m = 1, 2, \dots, L + 1$, $l = 1, 2, \dots, N$. Then, we can accumulate $g_{n,l,m}$ as an $M \times 1$ vector form $\mathbf{g}_{n,l} = [g_{n,l,1}, g_{n,l,2}, \dots, g_{n,l,M+1}]$. Then, the evaluation process of particle n is summarized as:

- 1) If the number of m in $g_{n,l,m} < \gamma_g$, denoted as N_m , is less than dimension D , $\theta_{n,l} = 1 - \frac{\sum_{m'=1}^{N_m} g_{n,l,m'}}{N_m \gamma_g}$, where m' is the index that within this range of N_m .
- 2) If the number of m in $g_{n,l,m} < \gamma_g$ is equal or larger than D , then, $\theta = 0.5$.

The reason of choosing this evaluation is when every sub-fitness value is bigger than a predefined threshold, e.g. $\gamma_g = (\frac{\epsilon_{BS}}{2})^2$, or the number of anchor nodes having smaller sub-fitness value is less than D , which means the global best is far from the UAV real position, or the particle falls into local optima. In these cases, the global best value should be mainly chosen from $\mathbf{rbest}$. Hence, the weighting factor θ is small. After determining θ , the combination set in (40) will become (42), which will be performed to determine $\mathbf{rbest}$.

$$\begin{aligned} \Theta_{g,n,l} = \{ & [x_{p,n,l}, y_{p,n,l}], [x_{g,l}, y_{g,l}], [x_{p,n,l}, y_{g,l}], \\ & [x_{g,l}, y_{p,n,l}], [(1 - \theta_{n,l})x_{p,n,l} + \theta_{n,l}x_{g,l}], \\ & ((1 - \theta_{n,l})y_{p,n,l} + \theta_{n,l}y_{g,l}) \} \end{aligned} \quad (42)$$

Secondly, when the sub-fitness value is smaller than the threshold, but the number of those smaller results is larger than D , which means that either the global best position is close to the real position, or the distance measurements are good enough to localize the UAV follower. In this case, we simply keep equal weight of global best and personal best in $\mathbf{rbest}$, and repeat the combination selection by (40).

Step 3. Find the Optimal Combination: After determining the weighting factor of each case of reference indicators, the next is to modify the indicator and obtain $\mathbf{rbest}$. From the above evaluation result, if $\theta = 0.5$, which means the personal best is as equal as the global best, and it is likely to obtain good estimation. In this case, the reference best is same as the selection result (41).

$$\mathbf{rb}_{n,l} = \mathbf{tb}_{n,l} \quad (43)$$

Otherwise, if the other two cases are evaluated, we will find the minimum two positions of $\Theta_{g,n,l}$, and denoted as $\mathbf{rb}_{n,l,1}$ and $\mathbf{rb}_{n,l,2}$ in (44).

$$\begin{aligned} \mathbf{rb}_{n,l,1} = \min_{i,j=1:2} \{ & f([x_{i,n,l}, y_{i,n,l}], [x_{i,n,l}, y_{i,n,l}]) \\ & \times \in \Theta_{g,n,l}, i, j \in \{p, g\} \} \\ \mathbf{rb}_{n,l,2} = \min_{i,j=1:2} \{ & f([x_{i,n,l}, y_{i,n,l}], [x_{i,n,l}, y_{i,n,l}]) \\ & \times \in \{\Theta_{g,n,l} \setminus \mathbf{rb}_{n,l,1}\}, i, j \in \{p, g\} \} \end{aligned} \quad (44)$$

Then, the $\mathbf{rbest}$ will be updated by a weighted combination of the two minimum results by

$$\mathbf{rb}_{n,l} = \theta_{n,l} \mathbf{rb}_{n,l,1} + (1 - \theta_{n,l}) \mathbf{rb}_{n,l,2} \quad (45)$$

to maintain the global best search ability of particles. And finally, the speed of particle will be updated by:

$$\mathbf{v}_{n,l} = \mathbf{v}_{n,l-1} + c\mathbf{r}(\mathbf{rb}_{n,l} - \mathbf{p}_{n,l}) \quad (46)$$

where $c = \frac{c_1 + c_2}{2}$ is the combined coefficient, $\mathbf{r}$ is $U(0, 1)$ random vectors, and the position of each particle will be updated by:

$$\mathbf{p}_{n,l} = \mathbf{p}_{n,l-1} + \mathbf{v}_{n,l} \quad (47)$$

E. Proposed PSO Design Part 3: Improving PSO Performance With Sufficient Particle

When the number of particles is sufficiently large, the particles are to be affected by their cognitive behaviors when the measurement error is large, which is corresponding to finding 4 in previous analysis. In this case, the global best may not be the expected position although it has the minimum fitness. The personal best will be more important. When $\frac{R}{2Q+1}$ particles are saved and utilized for the rest $N - Q$ iterations, the particles' cognitive behaviours in updating personal best should be chosen carefully, so that the particles will not converge toward the global best directly.

Moreover, comparing sub-fitness in this case is too costly. Therefore, we adaptively adjust c_1 and c_2 based on comparing the values of fitness function, and choose the weighted average as the reference best, whose corresponding fitness may not be the smallest, but is more likely to be closer to expected position. The overall flowchart of proposed PSO design in this section is summarized in Fig. 5, which is called RPSO type two (RPSO-II). Next we will explain the procedures step-by-step.

Step 1. Obtain Fitness Values: After the HPSO operation in the first Q iterations, the remaining number of particle is $R_{left} = \frac{R}{2Q+1}$. In each iteration $l = Q + 1, \dots, N$, the UAV leader will compute R_{left} fitness values by definition, denoted as $f_{n,l}$.

Step 2. Evaluate and Predict: The evaluation process for RPSO-II is briefly summarized as:

- 1) If $f_{n,l} \leq \gamma_n$, $\forall n$, then, jump out of RPSO, conventional PSO is utilized, $c_1 = c_2$ are unchanged.
- 2) If the number of particles in $S_n = \{n | f_{n,l} < \gamma_n\}$ is denoted as H_l , and $H_l < \frac{R}{2Q+1}$, or $f_{n,l} > \gamma_n$, $\forall n$, then, exploration stage is defined, i.e. global best weights more.
- 3) If $H_l > \frac{R}{2Q+1}$, then, exploitation stage is defined, i.e. personal best weights more.

This evaluation process means that if the fitness values are relatively low, the global best is a good estimation. In this way, there is no need to change c_1 and c_2 , the conventional PSO will be directly deployed. This stage usually happens in the last few iterations when particles are relatively stable.

However, when the fitness values of many or even all of the particles are higher than the threshold, the particles may have fallen into local optima. Then, the exploration stage is defined, which means the global best is still needed to be found before a stable stage comes. This stage usually happens in the beginning few iterations.

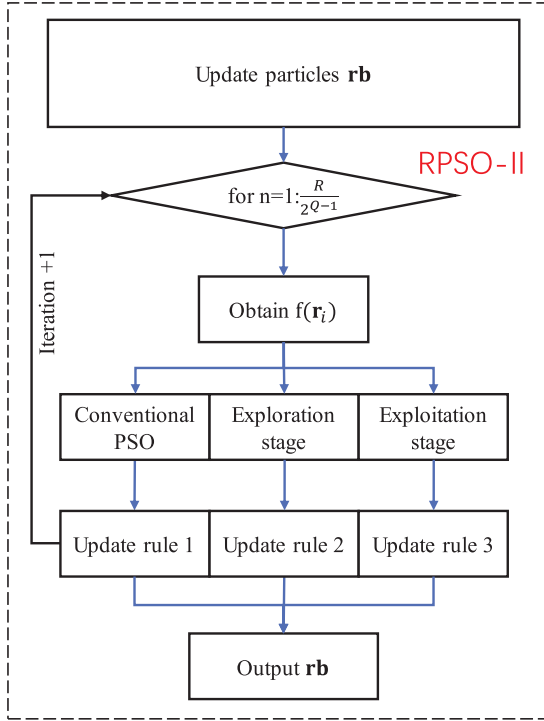


Fig. 5. Flowchart of RPSO-II.

Lastly, if more than $\frac{R}{2Q+1}$ particles are with lower fitness values, the particles may have fallen into a global optima. Although it is favored to be the stable situation, the global best may not be the expected position due to measurement error. In this way, the exploitation stage is defined, which means the personal best of each particle will have higher weight. This usually happens in the middle of iterations, or when the measurement error is large so that the particles are unstable. To tackle the problem of measurement error, we will take a weighed average as the reference best.

After the evaluation process, the prediction process is to determine c_1 and c_2 of next iteration. If RPSO is jumped out, then c_1 and c_2 remain unchanged. If exploration stage is defined, then c_1 is decreased and c_2 is increased.

$$c_{1,n,l} = c_1 - \alpha_{n,l} \quad (48)$$

$$c_{2,n,l} = c_2 + \alpha_{n,l} \quad (49)$$

where $\alpha_{n,l}$ is defined as:

$$\alpha_{n,l} = \frac{\sum_{n_\mu=1, n_\mu \notin S_n}^{R_{left}-H_l} f_{n_\mu,l}}{(R_{left} - H_l) \max_{n_\mu} f_{n_\mu,l}} \times \frac{\min_{j=1,2,\dots,l} f_{n,j}}{f_{n,l}} \quad (50)$$

On the other hand, if exploitation stage is defined, then c_1 is increased and c_2 is decreased:

$$c_{1,n,l} = c_1 + \beta_{n,l} \quad (51)$$

$$c_{2,n,l} = c_2 - \beta_{n,l} \quad (52)$$

where $\beta_{n,l}$ is defined as:

$$\beta_{n,l} = \frac{\min_{n_\tau \in S_n} f_{n_\tau,l} \times \min_{j=1,2,\dots,l} f_{n,j}}{f_{n,l}^2} \quad (53)$$

For exploration stage, the coefficient $\alpha_{n,l}$ is equivalent to the product of normalized mean fitness value of particles outside the set $S_n = \{n | f_{n,l} < \gamma_n\}$, and normalized personal best fitness. This coefficient can be adaptively controlled by the overall fitness and the personal fitness. If this particle is with higher fitness, then, $\alpha_{n,l}$ is relatively larger so the particle moves faster, and vice versa.

On the other hand, for exploitation stage, the coefficient $\beta_{n,l}$ is set as the product of the relative global best inside the set S_n , and the normalized personal best. This coefficient can conservatively make the particles having relatively low fitness to exploit toward different directions more frequently and slightly. In this way, the particles are relatively stable. But in order to deal with the measurement error, we will add a weighted average before particle updates.

Step 3. Update by Weighted Average: After determining the coefficients c_1 and c_2 , in next the particles will be updated according to this rule:

If the jump out stage is defined, the particles will be updated following the conventional PSO criteria in (14) (rule 1).

When the exploration stage is defined, the particle will be updated by using the reference best, denoted as

$$\mathbf{rb}_{n,l} = c_{1,n,l}(\mathbf{pb}_{n,l} - \mathbf{p}_{n,l}) + c_{2,n,l}(\mathbf{gb}_l - \mathbf{p}_{n,l}) \quad (54)$$

and the speed of this particle will be updated by $\mathbf{v}_{n,l} = \mathbf{v}_{n,l-1} + \mathbf{r} \cdot \mathbf{rb}_{n,l}$ until exploitation stage comes (rule 2).

When exploitation stage is defined, we use H_n fitness values to formulate a weighted average as the reference best:

$$\mathbf{rb}_{n,l} = \frac{\sum_{n_\tau=1}^{H_n} f_{n_\tau,l} \mathbf{tb}_{n_\tau,l}}{\sum_{n_\tau=1}^{H_n} f_{n_\tau,l}} \quad (55)$$

where $\mathbf{tb}_{n_\tau,l} = c_{1,n_\tau,l}(\mathbf{pb}_{n_\tau,l} - \mathbf{p}_{n_\tau,l}) + c_{2,n_\tau,l}(\mathbf{gb}_l - \mathbf{p}_{n_\tau,l})$ is the combination of $\mathbf{pbest}$ and $\mathbf{gbest}$ of H_n particles. The velocity of particle will be updated by (rule 3):

$$\mathbf{v}_{n,l} = \mathbf{v}_{n,l-1} + \mathbf{r} \cdot \mathbf{rb}_{n,l} \quad (56)$$

In this way, the particles in different stage are all updated accordingly. Note that the performance of proposed HRPSO will be similar to the convergence performance of standard PSO. This is because both RPSO-I and RPSO-II will have convergence due to the update rule in determining reference best. As HPSO only reduces particle number, the overall HRPSO will therefore converge.

F. Overall Flowchart of HRPSO

Eventually, the proposed PSO is denoted as HRPSO, and the overall flowchart of HRPSO is summarized as Fig. 6. On the complexity of overall HRPSO, it is still comparable or less than standard PSO. In HPSO, the load of calculating fitness is reduced. Therefore, the HPSO reduces complexity. For RPSO-I, choosing reference best from Θ increase the complexity to

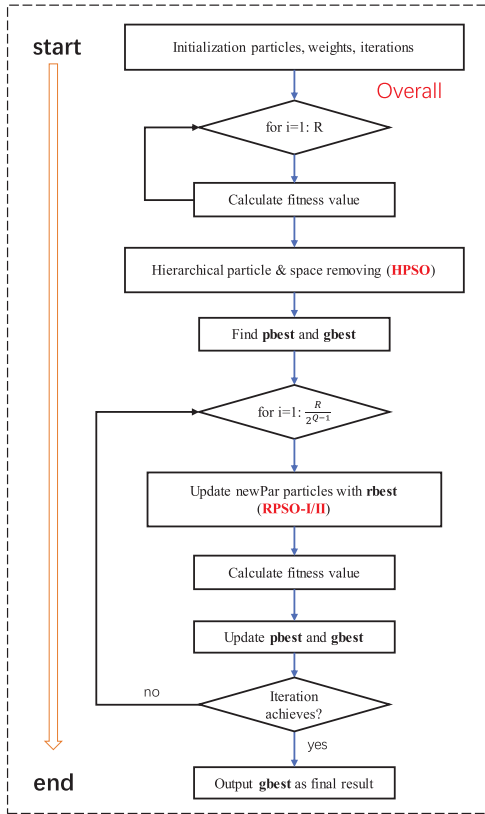


Fig. 6. Overall flowchart of proposed HRPSO.

four times particle number due to calculating the fitness in Θ . Since RPSO-I happens after HPSO reduces total number of particles by 2^{Q-1} times, RPSO-I will lead to similar complexity compared with standard PSO. On the other hand, for RPSO-II, the complexity will only be slightly increased. Therefore, the complexity of HPSO plus RPSO-II is still reduced compared with conventional PSO.

IV. SIMULATION RESULTS

A. HPSO Performance Comparison

Firstly, we validate the HPSO design and compare it with the conventional PSO on both search space and particle efficiency. $R = 256$ particles are initialized, the iteration time is set as $N = 20$ and $Q = 3$ for overall PSO iteration and the particle reduction iteration in HPSO. After 20 iterations, the search space of HPSO and conventional PSO are listed as Figs. 7 and 8. We can see from those two figures that the search space in HPSO is much smaller than that of conventional PSO after 20 iterations. Besides, after the first 3 iterations, the number of particles in HPSO is reduced to 64 instead of 256 in conventional PSO. Furthermore, we compare the particle efficiency in (19), and count the total particles consumed, where both R and N changes, while $Q = 3$ remains unchanged in HPSO. The results are listed in Figs. 9 and 10, respectively. We can clearly see the enhanced performance of HPSO compared with conventional PSO.

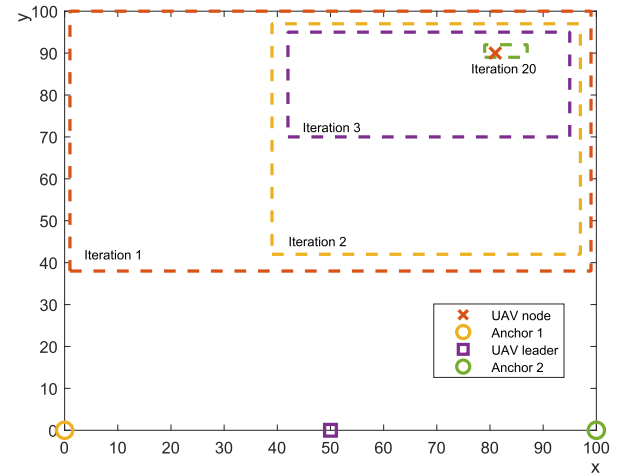


Fig. 7. Search space reduction result (HPSO) over 20 iterations.

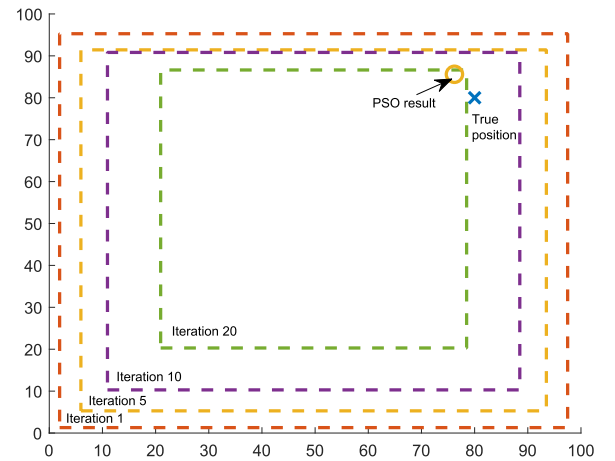


Fig. 8. Original PSO result over 20 iterations.

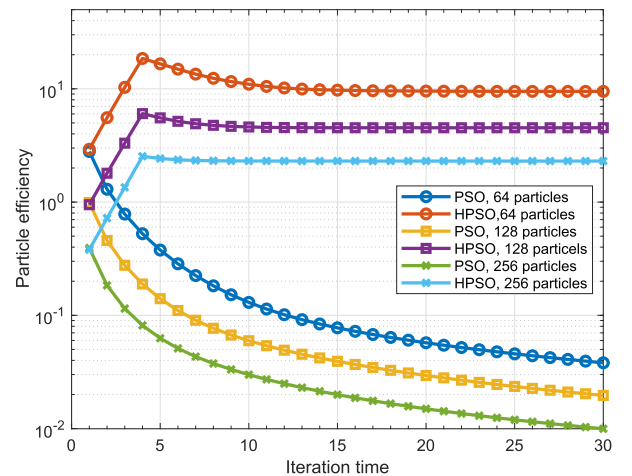


Fig. 9. Particle efficiency comparison.

B. Performance of RPSO Comparison

Now, we validated the proposed RPSO design and compare it with the conventional PSO [16]. Note that the particle number determines it is whether sufficient or insufficient. Choosing

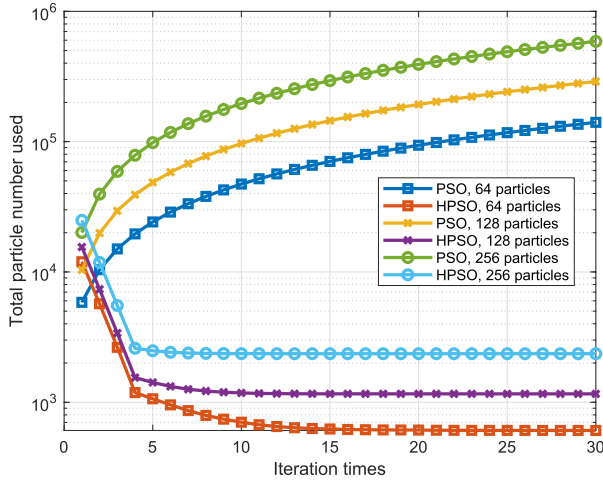


Fig. 10. Total consumed particle number comparison.

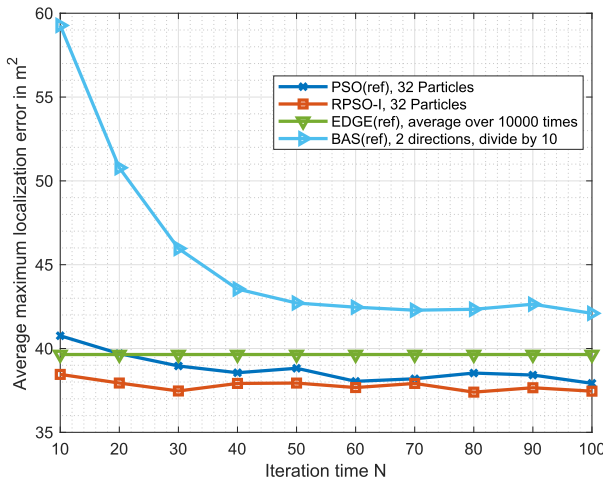


Fig. 11. RPSO-I localization error comparison versus iteration time.

proper particle number is an interesting topic to be further discussed. In this article, we select the particle number empirically. For the case then particle number is insufficient, we utilize RPSO-I when the particle number is equal or less than 64, which is found by empirical observation. When the particle number is larger than 64, RPSO-II will be deployed.

Firstly, we compare the localization error versus iteration time N for $R = 32$ particles in Fig. 11, the performance of RPSO-I is compared with conventional PSO, the Trilateration method (EDGE) and beetle antennae search (BAS) [34]. EDGE is the simplest yet most commonly used method, which performs well in low-error situations. BAS is a low complexity swarm intelligence algorithm that is recently proposed. From this figure, we can see the performance improvement in lower localization error of RPSO-I.

Note that both the proposed RPSO and standard PSO include particle update with randomness, now we compare the statistical results of RPSO-I, EDGE and standard PSO. The iteration number is fixed as $N = 20$. The distance measurement error variance in (9) are all set as $\sigma = 10$, so the threshold $\gamma_g = 25$.

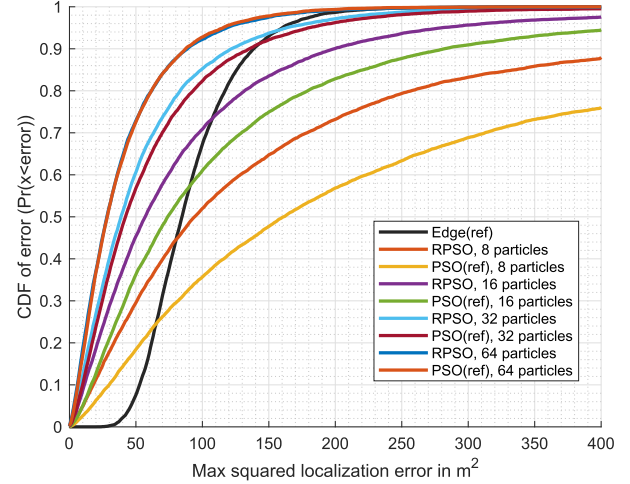


Fig. 12. CDF of localization error with distance measurement error.

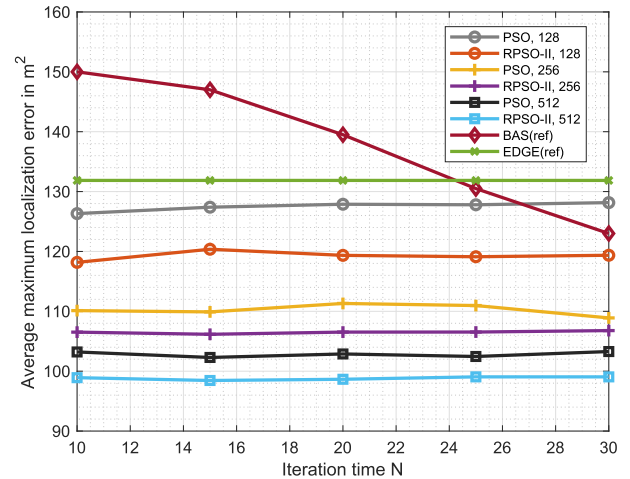


Fig. 13. RPSO-II localization error comparison versus iteration time.

The cumulative distribution function (CDF) of squared localization error is compared, where 100000 runs are recorded for different number of particles. We can find in Fig. 12 that RPSO-I performs better than conventional PSO and EDGE in CDF of localization error, which further validate the effectiveness of proposed method.

For the case of sufficient particle number, the RPSO-II is simulated for the average of the maximum squared localization error in order to compare the ability in exploration and exploitation, i.e. maintain better performance when distance measurement error is large. The iteration number varies from 10 to 30, and the particle numbers are set as 128, 256 and 512. Compared with conventional PSO, EDGE and BAS, our approach in Fig. 13 is better. Note that RPSO-II does not increase much computation load if Q iterations can be also performed at initialization stage.

Further, we validate the RPSO performance with varied distance measurement error variance σ . It is intuitive that with larger σ , the localization error will also be larger. Fig. 14 shows the result of mean squared-root localization error (in meters) of standard PSO and proposed RPSO-II, where 128 particles are utilized for 10, 20 and 30 iteration times.

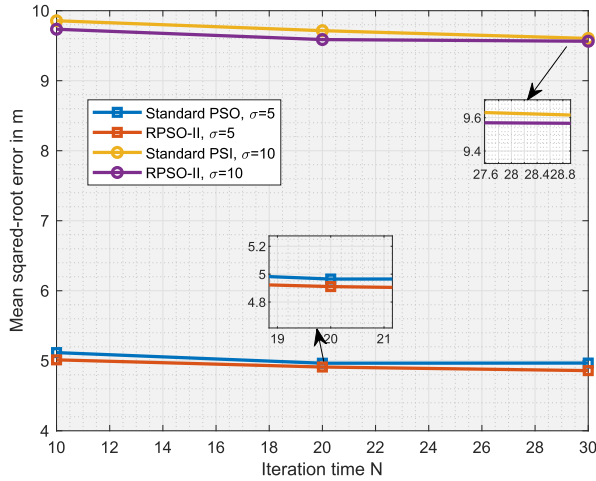


Fig. 14. RPSO-II localization error comparison versus distance measurement noise σ .

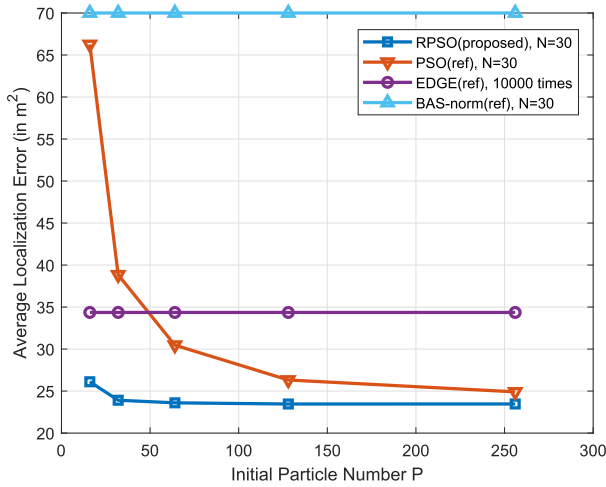


Fig. 15. Overall localization performance comparison versus particle number.

C. Performance of Overall HRPPO

Now, we demonstrate the localization error performance of overall proposed HRPPO versus PSO, EDGE and BAS. The initial particle number varies from 16 to 256, the iteration time $N = 30$. Note that RPSO-I is used for $R \leq 64$ and RPSO-II is used for $64 < R \leq 256$. HRPPO and PSO are tested for 10000 times to calculate the average, while EDGE and BAS are independently averaged over 10000 tests. The result of average localization error is in m^2 , which means the squared error of x plus y axis for $D = 2$. The result is shown in Fig. 15. We can find the effectiveness of proposed HRPPO over other methods.

V. CONCLUSION

Accurate localization is frequently required in UAV communication and related applications. In this article, we have proposed HRPPO to improve the performance of PSO utilized in UAV localization. HRPPO is firstly introduced to reduce the complexity by eliminating particles together with search space.

Then, HPSO is demonstrated to have lower computation load with higher particle efficiency. Further, RPSO is introduced for both insufficient and sufficient particles. The performance of RPSO is also shown to be better than conventional PSO and other methods.

REFERENCES

- [1] C. Zhang and W. Zhang, "Spectrum sharing for drone networks," *IEEE J. Sel. Areas Commun.*, vol. 35, no. 1, pp. 136–144, Jan. 2017.
- [2] M. E. M. Kramweni, C. Yang, J. Li, and W. Zhang, "A survey of game theory in unmanned aerial vehicles communications," *IEEE Commun. Surv. Tut.*, vol. 21, no. 4, pp. 3386–3416, Oct.–Dec. 2019.
- [3] J. A. J. Berni, P. J. Zarco-Tejada, L. Suarez, and E. Fereres, "Thermal and narrowband multispectral remote sensing for vegetation monitoring from an unmanned aerial vehicle," *IEEE Trans. Geosci. Remote Sens.*, vol. 47, no. 3, pp. 722–738, Mar. 2009.
- [4] Y. Zeng and R. Zhang, "Energy-efficient UAV communication with trajectory optimization," *IEEE Trans. Wireless Commun.*, vol. 16, no. 6, pp. 3747–3760, Jun. 2017.
- [5] Q. Zhang, M. Jiang, Z. Feng, W. Li, W. Zhang, and M. Pan, "IoT enabled UAV: Network architecture and routing algorithm," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 3727–3742, Apr. 2019.
- [6] B. Li, Z. Fei, and Y. Zhang, "UAV communications for 5G and beyond: Recent advances and future trends," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 2241–2263, Apr. 2019.
- [7] M. Mozaffari, W. Saad, M. Bennis, Y. -H. Nam, and M. Debbah, "A tutorial on UAVs for wireless networks: Applications, challenges, and open problems," *IEEE Commun. Surv. Tut.*, vol. 21, no. 3, pp. 2334–2360, Jul.–Sep. 2019.
- [8] L. Gupta, R. Jain, and G. Vaszkun, "Survey of important issues in UAV communication networks," *IEEE Commun. Surv. Tut.*, vol. 18, no. 2, pp. 1123–1152, Apr.–Jun. 2016.
- [9] C. Pan, R. Liu, and G. Yu, "Joint user association and resource allocation for mmWave communication: A neural network approach," *J. Commun. Inf. Netw.*, vol. 6, no. 2, pp. 125–133, Jun. 2021.
- [10] C. Zhang, W. Z. Zhang, W. Wang, L. Yang, and W. Zhang, "Research challenges and opportunities of UAV millimeter-wave communications," *IEEE Wireless Commun.*, vol. 26, no. 7, pp. 58–62, Feb. 2019.
- [11] Z. Xiao, H. Dong, L. Bai, D. O. Wu, and X. -G. Xia, "Unmanned aerial vehicle base station (UAV-BS) deployment with millimeter-wave beamforming," *IEEE Internet Things J.*, vol. 7, no. 2, pp. 1336–1349, Feb. 2020.
- [12] W. Wang and W. Zhang, "Jittering effects analysis and beam training design for UAV millimeter wave communications," *IEEE Wireless Commun.*, vol. 21, no. 5, pp. 3131–3146, May 2022.
- [13] W. Zhang, W. Zhang, and J. Wu, "UAV beam alignment for highly mobile millimeter wave communications," *IEEE Trans. Veh. Technol.*, vol. 69, no. 8, pp. 8577–8585, Aug. 2020.
- [14] J. Seo, Y. Kim, S. Kim, and A. Tsourdos, "Collision avoidance strategies for unmanned aerial vehicles in formation flight," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 53, no. 6, pp. 2718–2734, Dec. 2017.
- [15] H. Hu, K. Xiong, G. Qu, Q. Ni, P. Fan, and K. B. Letaief, "AoI-minimal trajectory planning and data collection in UAV-Assisted wireless powered IoT networks," *IEEE Internet Things J.*, vol. 8, no. 2, pp. 1211–1223, Jan. 2021.
- [16] M. Y. Arafat and S. Moh, "Localization and clustering based on swarm intelligence in UAV networks for emergency communications," *IEEE Internet Things J.*, vol. 6, no. 5, pp. 8958–8976, Oct. 2019.
- [17] J. N. Gross, Y. Gu, and M. B. Rhudy, "Robust UAV relative navigation with DGPS, INS, and peer-to-peer radio ranging," *IEEE Trans. Autom. Sci. Eng.*, vol. 12, no. 3, pp. 935–944, Jul. 2015.
- [18] A. Cho, J. Kim, S. Lee, and C. Kee, "Wind estimation and airspeed calibration using a UAV with a single-antenna GPS receiver and pitot tube," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 47, no. 1, pp. 109–117, Jan. 2011.
- [19] J. Farrell and T. Givargis, "Differential GPS reference station algorithm-design and analysis," *IEEE Trans. Control Syst. Technol.*, vol. 8, no. 3, pp. 519–531, May 2000.
- [20] S. Minaeian, J. Liu, and Y. Son, "Vision-based target detection and localization via a team of cooperative UAV and UGVs," *IEEE Trans. Syst. Man, Cybern. Syst.*, vol. 46, no. 7, pp. 1005–1016, Jul. 2016.

- [21] Q. Guo, Y. Zhang, J. Lloret, B. Kantarci, and W. K. G. Seah, "A localization method avoiding flip ambiguities for Micro-UAVs with bounded distance measurement errors," *IEEE Trans. Mobile Comput.*, vol. 18, no. 8, pp. 1718–1730, Aug. 2019.
- [22] Z. Zhang, K. Long, J. Wang, and F. Dressler, "On swarm intelligence inspired self-organized networking: Its bionic mechanisms, designing principles and optimization approaches," *IEEE Commun. Surv. Tut.*, vol. 16, no. 1, pp. 513–537, Jan.–Mar. 2014.
- [23] Javier Del Ser *et al.*, "Bio-inspired computation: Where we stand and what's next," *Swarm Evol. Comput.*, vol. 48, no. 1, pp. 220–250, Aug. 2019.
- [24] K. Gao, Z. Cao, L. Zhang, Z. Chen, Y. Han, and Q. Pan, "OA review on swarm intelligence and evolutionary algorithms for solving flexible job shop scheduling problems," *IEEE/CAA J. Autom. Sin.*, vol. 6, no. 4, pp. 904–916, Jul. 2019.
- [25] M. Dorigo, G. D. Caro, and L. M. Gambardella, "Ant algorithms for discrete optimization," *Artif. Life.*, vol. 5, no. 2, pp. 137–172, Apr. 1999.
- [26] Z. Zhan, J. Zhang, Y. Li, and Y. Shi, "Orthogonal learning particle swarm optimization," *IEEE Trans. Evol. Comput.*, vol. 15, no. 6, pp. 832–847, Dec. 2011.
- [27] E. Babaei Tirkolaee, A. Goli, and G. -W. Weber, "Fuzzy mathematical programming and self-adaptive artificial fish swarm algorithm for just-in-time energy-aware flow shop scheduling problem with outsourcing option," *IEEE Trans. Fuzzy Syst.*, vol. 28, no. 11, pp. 2772–2783, Nov. 2020.
- [28] Y. Wang, W. Chen, and C. Tellambura, "A PAPR reduction method based on artificial bee colony algorithm for OFDM signals," *IEEE Wireless Commun.*, vol. 9, no. 10, pp. 2994–2999, Oct. 2010.
- [29] J. Zhao, S. Liu, M. Zhou, X. Guo, and L. Qi, "Modified cuckoo search algorithm to solve economic power dispatch optimization problems," *IEEE/CAA J. Autom. Sin.*, vol. 5, no. 4, pp. 794–806, Jul. 2018.
- [30] A. H. Khan, X. Cao, S. Li, V. N. Katsikis, and L. Liao, "BAS-ADAM: An ADAM based approach to improve the performance of beetle antennae search optimizer," *IEEE/CAA J. Autom. Sin.*, vol. 7, no. 2, pp. 461–471, Mar. 2020.
- [31] R. Liu, Y. Ma, X. Zhang, and Y. Gao, "Deep learning-based spectrum sensing in space-air-ground integrated networks," *J. Commun. Inf. Netw.*, vol. 6, no. 1, pp. 82–90, Mar. 2021.
- [32] H. Zhang, L. Song, and Z. Han, *Unmanned Aerial Vehicle Applications Over Cellular Networks for 5G and Beyond*. Berlin, Germany: Springer, 2020.
- [33] J. Hu, H. Zhang, L. Song, Z. Han, and H. Poor, "Reinforcement learning for a cellular internet of UAVs: Protocol design, trajectory control, and resource management," *IEEE Wireless Commun.*, vol. 27, no. 1, pp. 116–123, Feb. 2020.
- [34] X. Jiang and S. Li, "BAS: Beetle antennae search algorithm for optimization problems," *Int. J. Robot. Control.*, vol. 1, no. 1, pp. 1–5, 2018.



layer machine learning.

Weizheng Zhang (Member, IEEE) received the B.Eng. degree in information science and engineering from Southeast University, Nanjing, China, in 2015, and the Ph.D. degree in electrical engineering from the University of New South Wales, Sydney, NSW, Australia, in 2019, respectively. He is currently a Postdoctoral Research Fellow with the Dongguan University of Technology, Dongguan, China, and the University of New South Wales. His research interests include millimetre-wave communication, signal processing for UAV communications, and physical



Wei Zhang (Fellow, IEEE) received the Ph.D. degree from The Chinese University of Hong Kong, Hong Kong, in 2005. He is currently a Professor with the School of Electrical Engineering and Telecommunications, University of New South Wales, Sydney, NSW, Australia. His research interests include UAV communications, 5G and beyond. He was the recipient of six best article awards from IEEE conferences and ComSoc technical committees. He was an IEEE ComSoc Distinguished Lecturer during 2016–2017.

He is a Vice President of IEEE Communications Society. Within the IEEE ComSoc, he has taken many leadership positions, including Member-at-Large on the Board of Governors during 2018–2020, the Chair of Wireless Communications Technical Committee during 2019–2020, Vice Director of Asia Pacific Board during 2016–2021, Editor-in-Chief of IEEE WIRELESS COMMUNICATIONS LETTERS during 2016–2019, Technical Program Committee Chair of APCC 2017 and ICC 2019, Award Committee Chair of Asia Pacific Board, and Award Committee Chair of Technical Committee on Cognitive Networks. In addition, he was a Member in various ComSoc boards/standing committees, including Journals Board, Technical Committee Recertification Committee, Finance Standing Committee, Information Technology Committee, Steering Committee of IEEE TRANSACTIONS ON GREEN COMMUNICATIONS AND NETWORKING and Steering Committee of *IEEE Networking Letters*. He is also the Area Editor of the IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS and the Editor-in-Chief of *Journal of Communications and Information Networks*. He was the Editor of the IEEE TRANSACTIONS ON COMMUNICATIONS, IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS, IEEE TRANSACTIONS ON COGNITIVE COMMUNICATIONS AND NETWORKING, and IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS-COGNITIVE RADIO SERIES.